BM25 Retrieval and LambdaMART Re-ranking for LinkedIn Skills/Job Matching

Patrick Gomes de Oliveira

October 15, 2026

Abstract

We evaluate a classical, fully non-neural two-stage retrieve-then-rank pipeline – BM25 lexical retrieval followed by LambdaMART (LightGBM `lambdarank`) re-ranking over handcrafted features – for matching candidate skill profiles to job postings. The system includes point-in-time-safe features, a documented self-supervised proxy for the missing candidate-job interaction signal, taxonomy-normalized skill overlap, strong non-neural baselines, reproducible Floci/AWS infrastructure, and offline-to-online serving parity tests.

1 Introduction

Before neural dense retrieval, production search and recommendation systems were built on a two-stage pattern: a cheap lexical or statistical retriever narrows a large catalogue to a shortlist, and a boosted-tree ranker – trained on explicit, auditable features – reorders that shortlist for quality. This project reproduces that pattern end to end for matching candidate skill profiles to LinkedIn-style job postings: BM25 lexical retrieval (stage 1) followed by LambdaMART re-ranking (stage 2) on handcrafted, point-in-time-safe features. The scope is deliberately classical: no neural network, no learned embedding space, and no deep retriever appear anywhere in the stack. The central lesson is that a fully lexical/statistical pipeline remains a strong, inexpensive, and highly interpretable baseline, and that taxonomy normalization – collapsing skill synonyms to one canonical token – is the single engineering lever available to such a system in place of a learned embedding space.

2 Dataset and Proxy Supervision

The underlying data (the Kaggle release `asaniczka/1-3m-linkedin-jobs-and-skills-2024`, or a synthetic fixture with the same schema for credential-free CI) is job postings: a title, a free-text description, a required-skills list, a seniority level, a location, and a posting time. It is *not* a candidate-job interaction log: there are no real candidate profiles and no record of which postings a real candidate ever matched against. Evaluating a retrieval-and-ranking system therefore requires a documented proxy.

Each job posting's own canonical skill set stands in for a candidate profile that plausibly matches that job: a uniformly sampled 50-90% subset of its canonical skills is kept as the profile query, and the subsampling is what keeps the query from being a trivial verbatim duplicate of the posting. That profile is paired against its own posting as the single positive example, and against nine other randomly sampled postings as negatives, all grouped under the originating posting for LightGBM's `lambdarank` group structure. The originating posting's own time is treated as the group's query time, so a temporal split on it keeps an entire group inside one partition and still supports a genuine point-in-time cut-off: if τ_1 and τ_2 are the 80th and 90th percentiles of query time, training uses $t < \tau_1$, validation uses $\tau_1 \leq t < \tau_2$, and test uses $t \geq \tau_2$. A second, grouped random split (ignoring time entirely) is kept as a sanity check that the headline numbers are not an artifact of the temporal boundary.

Limitations. This construction can demonstrate that a two-stage system recovers a posting from a partial, resampled view of its own requirements, and that taxonomy normalization helps it do so. It cannot demonstrate that the system would satisfy a real candidate's preferences, since no real candidate signal exists in the underlying data. Real posting-time columns are also absent from the public Kaggle release; when `--kaggle` is used, a posting time is synthesized uniformly over a plausible scraping window and this is reported, not concealed, in every table this project generates.

3 Feature Engineering

The declarative catalogue in `conf/features.yaml` covers the candidate profile (skill count, seniority, taxonomy breadth), the candidate job (category popularity, trailing posting volume), the profile-job interaction (the stage-1 BM25 score itself, raw and taxonomy-normalized skill overlap, seniority match), and context (location match, a point-in-time posting-recency gap). Every aggregate over training postings is filtered with a strict $t <$ as_of comparison, never $\leq$: an event exactly at the cut-off belongs to

Table 1: Engineered features. Every aggregate over training postings is computed point-in-time: only events strictly before the prediction cut-off contribute to its value.

Feature	Family	PIT
`profile_skill_count`	user	–
`profile_seniority_level`	user	–
`profile_taxonomy_coverage`	user	–
`candidate_job_popularity_rank`	item	–
`candidate_job_trailing_postings`	statistical	✓
`bm25_score`	interaction	–
`skill_overlap_raw`	interaction	–
`skill_overlap_jaccard`	interaction	–
`seniority_match`	interaction	–
`location_match`	context	–
`query_candidate_recency_gap_days`	context	✓

the future partition. Schema, leakage, and offline/online parity tests enforce this contract, including a dedicated control that forges a future query and asserts the fitted item-level features do not move. The generated feature catalogue, with its declared rationale for each feature, is shown below.

4 Methodology

4.1 Stage 1: BM25 Retrieval

Given a query q (the candidate-profile skill tokens) and a document d (a job posting's title, skills, seniority, and location), BM25 scores

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{IDF}(t)\, \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1\big(1 - b + b\,\frac{|d|}{\mathrm{avgdl}}\big)},$$

where $f(t,d)$ is term t's frequency in d, $|d|$ is document length, avgdl is the mean document length over the indexed corpus, and k_1, b are the usual saturation and length-normalization constants. The inverse document frequency term is

$$\mathrm{IDF}(t) = \ln\!\left(\frac{N - n(t) + 0.5}{n(t) + 0.5} + 1\right),$$

with N indexed documents and $n(t)$ the number containing t. The index is fit on exactly the training partition's candidate pool (`rank_bm25`), so stage-1 retrieval itself obeys the `fit_on: train_only` contract.

4.2 The Taxonomy-Normalized Overlap Feature

BM25 rewards surface-form overlap: a query token must match a document token exactly. Skill mentions in practice are not canonical ("JS", "Javascript", "ECMAScript" are one skill under three spellings), so a taxonomy-normalized Jaccard overlap is computed separately and fed to stage 2 as a feature:

$$\mathrm{overlap}(C,J) = \frac{|\mathrm{skills}(C) \cap \mathrm{skills}(J)|}{|\mathrm{skills}(C) \cup \mathrm{skills}(J)|},$$

where skills$(\cdot)$ denotes the canonical (synonym-collapsed) skill set of a candidate profile C or job posting J. This is the one feature a purely lexical retriever cannot express, and it is the single strongest stage-2 signal by gain-based importance.

4.3 Stage 2: LambdaMART Re-ranking

LambdaMART optimizes a pairwise surrogate for a listwise ranking metric. For a pair of documents i (more relevant) and j (less relevant) within the same query group, the gradient of the loss with respect to the model's score s_i is

$$\lambda_{ij} = -\frac{\partial \mathcal{L}}{\partial s_i} = \sigma\big(s_j - s_i\big) \cdot |\Delta\mathrm{NDCG}_{ij}|,$$

where $\sigma(\cdot)$ is the logistic function and $|\Delta\mathrm{NDCG}_{ij}|$ is the change in NDCG that would result from swapping i and j in the current ranking. Scaling the pairwise gradient by this swap cost is what makes LambdaMART a listwise approximation rather than a plain pairwise classifier: mis-ordering two documents near the top of the ranked list is penalized far more than mis-ordering two documents far down it, which directly targets the NDCG objective stage 2 is evaluated against. LightGBM's `objective=lambdarank` trains exactly this gradient over the handcrafted feature matrix described in Section 3, using the LTR group structure from the proxy supervision in Section 2.

5 Experimental Setup

Every model is evaluated over the same candidate pool per partition (the distinct job postings that appear as a candidate anywhere in that partition's pairs) and the same cut-off K, so Recall@K, NDCG@K and MAP@K are directly comparable across models:

$$\mathrm{NDCG@}K = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{\sum_{j=1}^{K} rel_{qj} / \log_2(j+1)}{\mathrm{IDCG}_q@K},$$

where $rel_{qj} \in \{0,1\}$ is the proxy relevance label at rank j for query q (exactly one posting is relevant per query, by construction of the proxy supervision). Recall@K measures only stage 1: whether the true posting survived BM25 retrieval into the top-K at all, independent of its rank. NDCG@K and MAP@K are computed on whatever stage 1 already retrieved, which is what decomposes the end-to-end number into a coverage term and a ranking-quality term. Two non-neural baselines bound the comparison: a *popularity* baseline with no retrieval step (ranks the full pool by raw skill-overlap count), and

a *lexical-only* baseline with retrieval but no re-ranking (BM25 top-K, truncated). Hyperparameters for LightGBM are fixed rather than tuned, since the fixture's purpose is to exercise the pipeline and its leakage controls, not to report a state-of-the-art number. Synthetic data is used for every result in this report; real Kaggle data is downloaded on demand and is never committed.

6 Results

Table 2: Stage-1-vs-stage-2 comparison on the reproducible fixture (test partition).

Model	Recall@10	NDCG@10	MAP@10
Popularity	0.3733	0.2232	0.1769
LexicalOnly	0.9933	0.9128	0.8861
TwoStage	0.9933	0.9556	0.9422

The two-stage system (BM25 retrieval followed by LambdaMART re-ranking) matches or exceeds the lexical-only baseline on every metric and substantially exceeds the retrieval-free popularity baseline, confirming the decomposition the evaluation protocol is designed to expose: stage 1 alone already recovers most of the achievable recall on this fixture, and stage 2's contribution is concentrated in ranking quality (NDCG@K, MAP@K) rather than coverage. Gain-based feature importance consistently ranks the taxonomy-normalized skill overlap above the raw overlap feature, which is the quantitative counterpart to the qualitative claim in Section 4: lexical matching alone under-counts true overlap whenever a skill is spelled differently, and taxonomy normalization recovers exactly that gap.

7 Deployment

Training writes the BM25 index, the LightGBM model, and feature parquet to S3, and publishes precomputed top-N job matches to DynamoDB, keyed by profile. A Lambda behind API Gateway performs a bounded key lookup rather than scoring at request time, which keeps p99 latency bounded by DynamoDB rather than by BM25 index size or LightGBM inference cost. EventBridge schedules nightly retraining and CloudWatch receives logs. The default Terraform profile targets the local Floci AWS emulator and therefore requires no paid AWS account; flipping a single variable (`use_floci=false`) retargets the identical configuration at a real account.

8 Limitations and Conclusion

The central limitation is the one documented in Section 2: there is no real candidate-job interaction signal, only a self-supervised proxy built from each posting's own (partially resampled) requirements. The reported numbers demonstrate that the pipeline mechanics are sound and leakage-free, not that the system would satisfy a real candidate's job search. Both BM25 and LambdaMART are also purely lexical and statistical: neither captures semantic skill similarity outside the hand-built taxonomy (a candidate strong in PyTorch but not listing "machine learning" verbatim gets no credit a learned embedding space would have given them), which is precisely the capability a neural dense retriever would add on top of this foundation. The project's contribution is therefore not a claim that lexical retrieval plus boosted-tree re-ranking is sufficient for production job matching, but a demonstration of the classical, production-proven predecessor to neural dense retrieval: a leakage-resistant, fully tested two-stage pipeline where every feature is auditable, every split is point-in-time safe, and the one lever available in place of a learned embedding space – taxonomy normalization – is shown, quantitatively, to matter.

References

[1] Christopher J.C. Burges. From ranknet to lambdarank to lambdamart: An overview. In *Microsoft Research Technical Report MSR-TR-2010-82*, 2010.

[2] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[3] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. In *Foundations and Trends in Information Retrieval*, volume 3, pages 333–389, 2009.